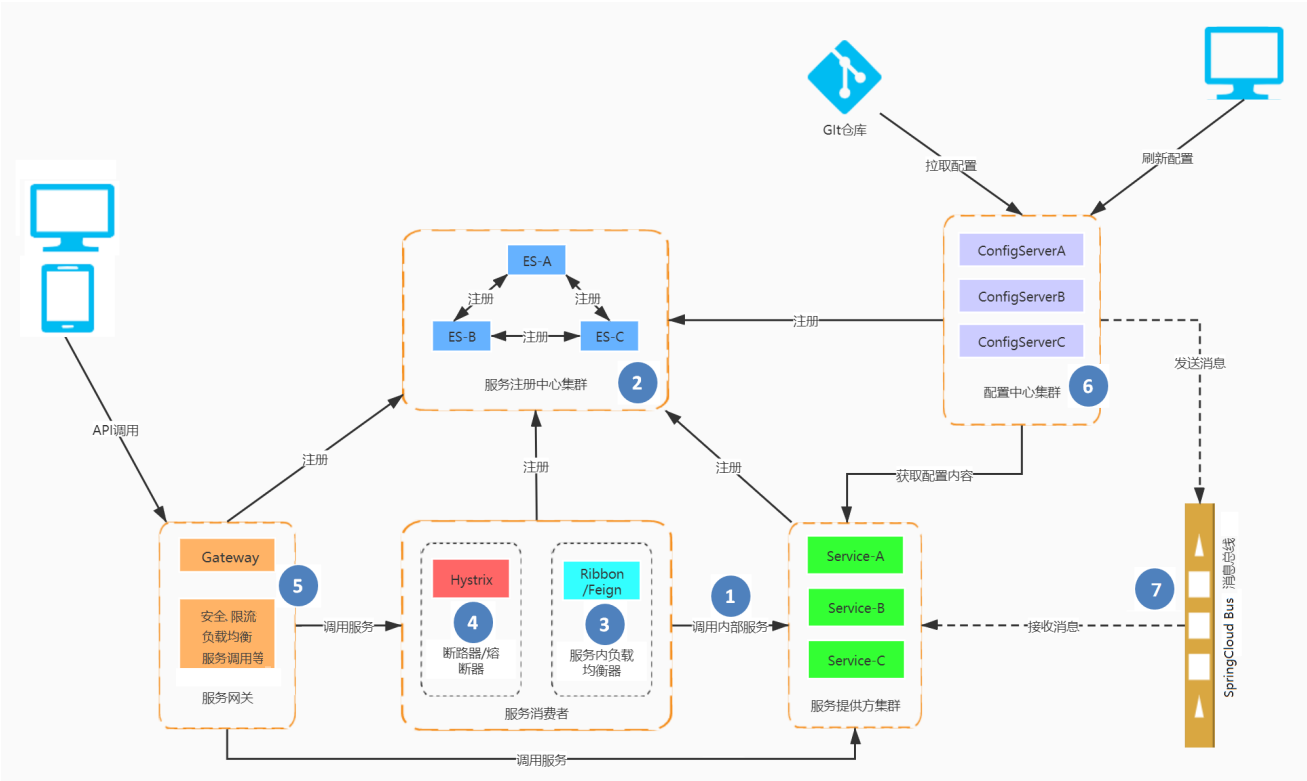
本章所讲内容为 Spring Cloud 架构图中的第四个部分。

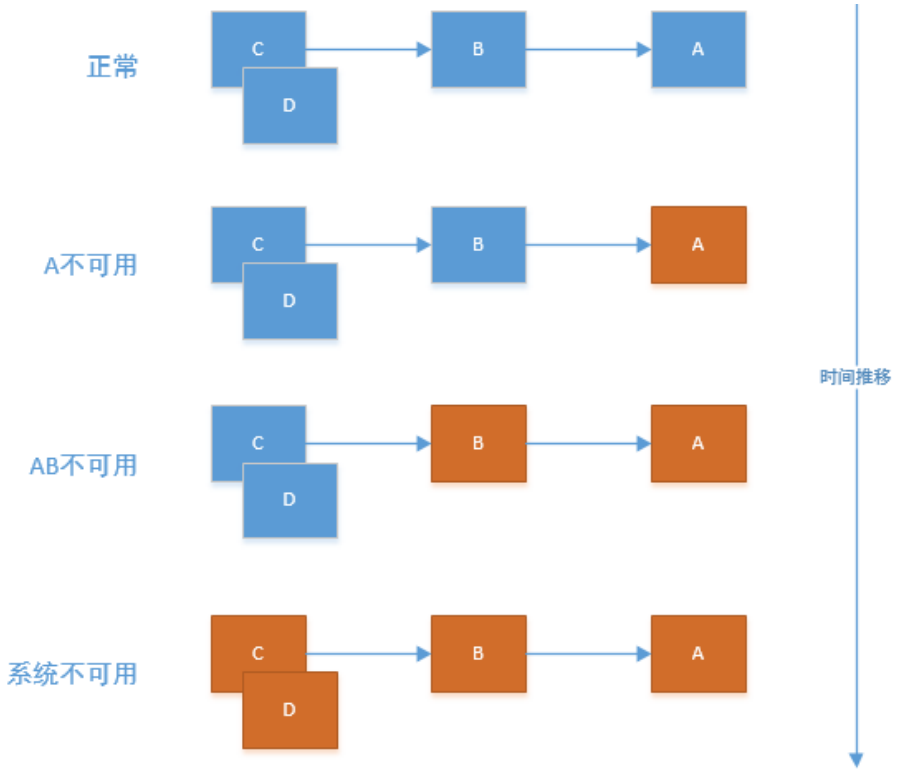



# 1.服务的延迟和容错

## 1.1.雪崩效应

在微服务架构中通常会有多个服务层调用，基础服务的故障可能会导致级联故障，进而造成整个系统不可用的情况，这种现象被称为服务雪崩效应。

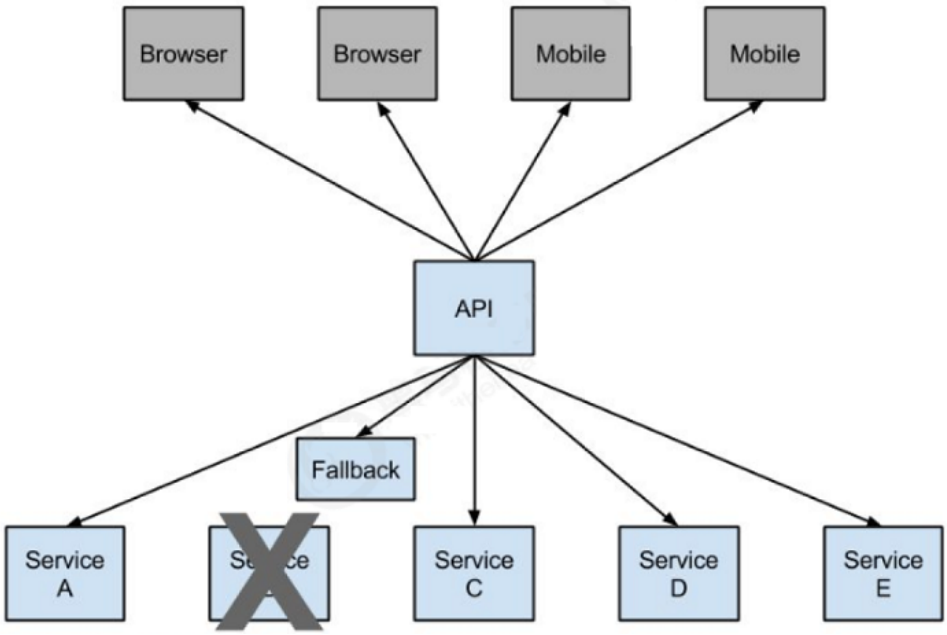
如果下图所示：A作为服务提供者，B为A的服务消费者，C和D是B的服务消费者。A不可用引起了B的不可用，并将不可用像滚雪球一样放大到C和D时，雪崩效应就形成了。




> 在微服务架构中，雪崩效应是一种因 “服务提供者” 的不可用导致 “服务消费者” 的不可用，并将不可用逐渐放大的一种蝴蝶效应。

导致雪崩效应发生的原因多种多样，有不合理的容量设计，或者是高并发下某一个方法响应变慢，亦或是某台机器的资源耗尽。从源头上我们无法完全杜绝雪崩源头的发生，但是雪崩的根本原因来源于服务之间的强依赖，所以我们可以提前评估，做好熔断、降级、隔离与限流。

## 1.2.熔断、降级、隔离与限流




1. 服务熔断

服务熔断是应对雪崩效应的一种微服务链路保护机制。当调用链路中的某个服务不可用或响应太慢时，会进行服务熔断，即停止该节点的调用，这样能够起到快速失败的目的。当检测到该节点服务恢复正常后，在恢复调用链路。这种牺牲局部，保全整体的措施就是熔断。

2. 服务降级

   当服务器压力剧增的情况下，根据实际业务情况及流量，对一些服务有策略的不处理或简单处理，从而释放资源，以保证核心业务的正常运行。比如：双11时，会把与购物无关的服务统统降级（历史订单，商品评论，只显示最后100条等等）。

   降级分类一般有：超时降级、失败次数降级、故障降级、限流降级等。

3. 服务隔离

   顾名思义，它是指将系统按照一定的原则划分为若干个服务模块，各个模块之间相对独立，无强依赖。当有故障发生时，能将问题和影响隔离在某个模块内部，而不扩散风险，不波及其它模块，不影响整体的系统服务。

   实际上，服务隔离说到底还是分治思想的体现。

4. 服务限流

   限流的目的是通过对并发请求进行限制，或者一个时间窗口内的请求进行限制来保护系统。一旦达到限流阈值则可以拒绝服务，或排队、等待、降级等。

# 2.Hystrix延迟和容错库

## 2.1.Hystrix简介

Hystrix是由Netflix开源的一个延迟和容错库，用于隔离访问远程系统、服务或者第三方库，防止级联失败，从而提升系统的可用性与容错性。也就是说：Hystrix实现了熔断、降级、隔离和限流。

Hystrix主要通过以下几点实现延迟和容错。

1. 包裹请求：使用HystrixCommand包裹对服务的请求，每个命令在独立线程中执行。
2. 回退机制：当请求失败、超时、被拒绝，或当断路器打开时，执行回退逻辑。回退逻辑由开发人员自行提供，例如返回一个缺省值。
3. 资源隔离：Hystrix为每个依赖都维护了一个小型的线程池（或者信号量）。如果该线程池已满，发往该依赖的请求就被立即拒绝，而不是排队等待，从而加速失败判定。
4. 熔断机制：当某服务的错误率超过一定的阈值时，Hystrix可以自动或手动跳闸，停止请求该服务一段时间。
5. 自我修复：断路器打开一段时间后，会自动进入“半开”状态。
6. 监控机制：Hystrix可以近乎实时地监控运行指标，例如成功、失败、超时、以及被拒绝的请求等。

## 2.2.使用Hystrix实现服务熔断

可以使用RestTemplate方式来实现Hystrix的服务熔断，也可以使用Feign方式实现Hystrix的服务熔断。

因为Feign已经整合了Hystrix，使用起来更加的方便，所以下面在consumer_server_12000工程中使用Feign实现服务熔断。

### 2.2.1.导入Hystrix依赖

SpringCloud默认已为Feign整合了hystrix，所以添加Feign依赖后就不用在添加hystrix依赖了。

> 可以看到依赖中已经有了feign-hystrix-x.jar文件，也可以在feign依赖中追查到hystrix依赖。

### 2.2.2.在Fegin中开启hystrix

修改consumer_server_12000工程的application.yml文件，开启hystrix熔断机制

```yaml
feign:
    hystrix:
        enabled: true        #在feign中开启hystrix熔断机制
```

### 2.2.3.创建FeignClient接口实现类

创建FeignClient接口实现类，目的就是在实现类中书写回退逻辑。

```java
package com.neusoft.feign;

import com.neusoft.po.User;

@Component
public class UserFeginClientCallBack implements UserFeginClient{

    //熔断降级方法
    @Override
    public CommonResult getUserById(Integer userId) {
        //403: 发送的请求被服务器拒绝
        return new CommonResult(403,"fegin触发了熔断降级方法",null);
    }
}
```

> 注意：要使用@Component注解将此实现类实例放入Spring容器中。

### 2.2.4.修改FeignClient接口配置

在FeignClient接口中，配置熔断发生时要调用的回退方法

```
//@FeignClient注解的fallback属性指定回退方法
@FeignClient(name="provider-server",fallback=UserFeginClientCallBack.class)
public interface UserFeginClient {

    //配置需要调用的挂号服务接口。与11000的UserController中的方法定义一致
    @GetMapping("/user/getUserById/{userId}")
    public User getUserById(@PathVariable("userId") Integer userId);
}
```

### 2.2.5.测试

- 服务不可用熔断测试：停止服务提供者（provider_server_11000、provider_server_11001），也就是模拟服务提供者宕机（也可以在服务提供者设置异常，来模拟服务提供者出现异常）。此时，当访问服务消费者（consumer_server_12000）时，就会看调用了熔断降级方法返回的信息。
- 服务超时熔断测试：在服务提供者中的控制器方法中，设置间隔1秒之后再响应（hystrix默认超时时间：1秒）。此时，也会看调用了熔断降级方法返回的信息。
- 服务异常熔断测试：在服务提供者中的控制器方法中，运行时如果出现异常，此时，也会看调用了熔断降级方法返回的信息。

# 3.Hystrix工作流程总结

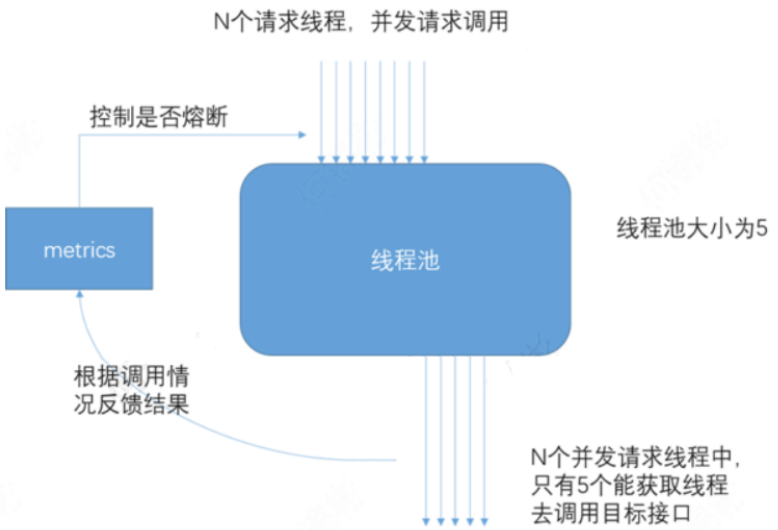
下面对Hystrix是如何实现隔离、限流、熔断、降级的，以及Hystrix的整体工作流程做一个总结。

## 3.1.Hystrix隔离与限流

Hystrix支持的隔离策略有两种：

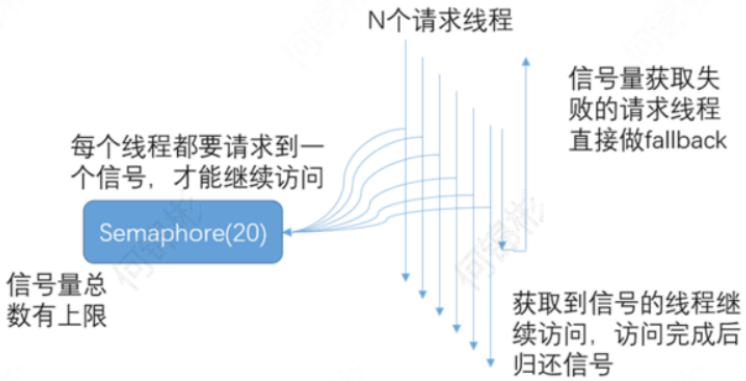
- 线程池隔离：为每个服务设置自己的请求线程池，每个线程池之间互不干涉。这样，某个服务的高延迟只会拖慢这个服务对应的线程池，而不会影响其他服务。

  缺点是有一定的资源消耗。优点是可以应对突发流量（当流量洪峰来临时，可以将请求放入线程池中慢慢处理）



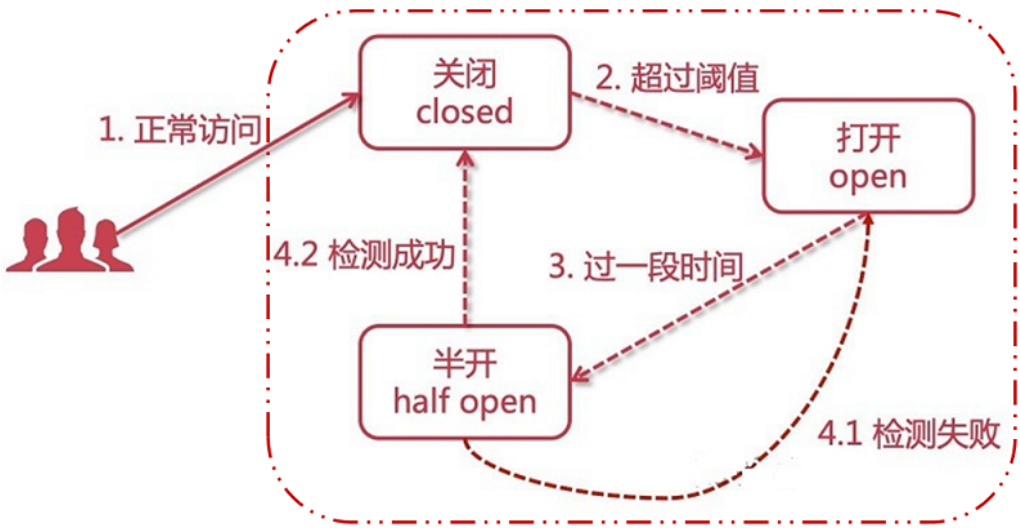

- 信号量隔离：使用计数器记录请求数量，超过阈值的请求直接返回。

  此种方式是严格的控制请求数量且立即返回模式。它耗费资源少，但无法应对突发流量（当流量洪峰来临时，超过阈值的请求会直接返回，不继续请求所依赖的服务）




## 3.2.Hystrix熔断与降级

Hystrix是通过断路器组件实现服务熔断与降级的。下面是Hystrix断路器的工作状态图：



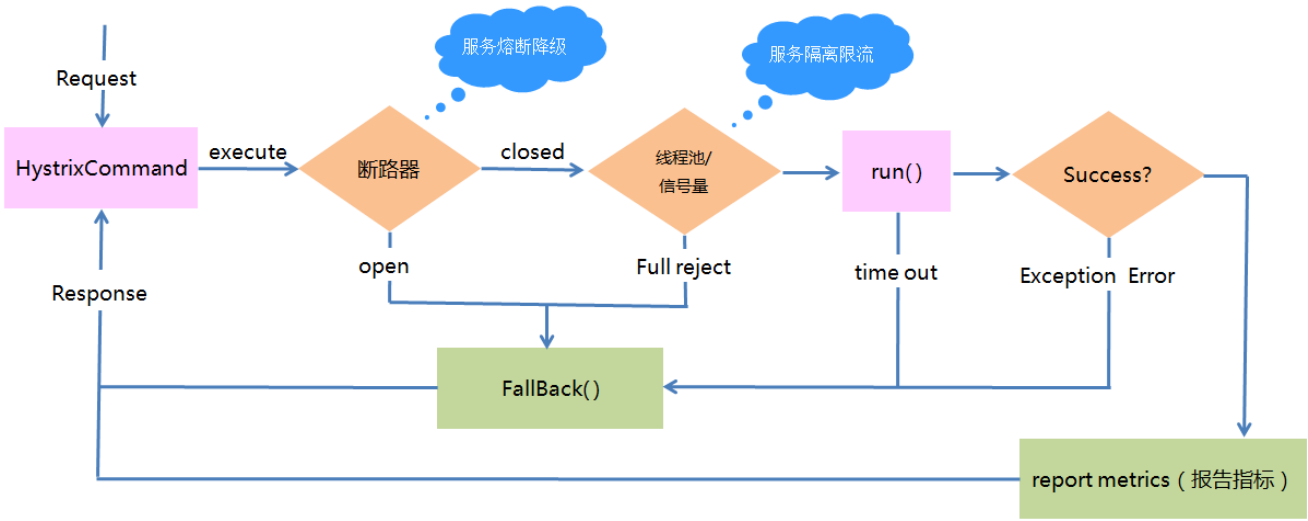

Hystrix断路器有三个状态 CLOSED 、OPEN 、HALF_OPEN：

- Closed（关闭状态、默认）：所有请求都正常访问。
- Open（打开状态）：所有请求都会被降级。Hystix会对请求情况计数，当一定时间内失败请求百分比达到阈值，则触发熔断，断路器会完全打开。
- Half Open（半开状态）：进入打开状态5秒后，断路器自动进入半开状态。此时断路器会尝试释放1次请求，若这个请求是健康的，则会关闭断路器；否则继续保持打开，再次进行5秒休眠计时。

Hystrix断路器打开的条件：

- 请求达到阈值，默认统计10秒内20次请求，如果失败率达到50%，则开启断路器。
- 信号量/线程池：通过度量桶（report metrics），检测信号量线程池是否已满，如果不能执行command，则开启断路器。
- run执行：通过度量桶（report metrics），检测成功失败信息，如果失败，则开启断路器。

## 3.3.Hystrix工作流程总结




1. 构建一个 HystrixCommand 对象，用于封装请求，并在构造方法配置请求被执行需要的参数。
2. 执行命令，Hystrix 提供了几种执行命令的方法，比较常用到的是 Synchrous 和 Asynchrous。
3. 判断电路是否被打开，如果被打开，直接进入 Fallback 方法。
4. 判断线程池/队列/信号量是否已经满，如果满了，直接进入 Fallback 方法。
5. 执行 Run 方法，一般是 HystrixCommand.run()，进入实际的业务调用。
6. 执行超时或者执行失败抛出未提前预计的异常时，直接进入 Fallback 方法。
7. 无论中间走到哪一步都会进行上报 Metrics，统计出熔断器的监控指标。
8. 最后返回响应。